



Project Title

EXPLAINABLE AI-DRIVEN LIGHTWEIGHT NETWORK INTRUSION DETECTION
SYSTEM

Muhammed Bertuğhan Kılınç-19COMP1102 Ayşe Korkut-21COMP1011 Mert Özbay-
21COMP1023

SYSTEM DESIGN DOCUMENT

SYSTEM DESIGN DOCUMENT

The System Design Document (SDD) is written after the initial system decomposition is done, and updated throughout the development. SDD describes the services provided by each subsystem. The services of subsystems serve as a reference for teams to clearly identify the boundaries between their subsystems and enables them to develop subsystems in parallel.

SDD is used to define interfaces between teams of developers and serve as a reference when architecture-level decisions need to be revisited. The audience for the SDD includes the project management, the system architects (i.e., the developers who participate in the system design), and the developers who design and implement each subsystem.

1. Introduction

This System Design Document (SDD) provides a comprehensive architectural overview of the AI-Driven Lightweight Network Intrusion Detection System. The primary purpose of this document is to describe the system's subsystems, their interfaces, and the overall software architecture. It serves as a core reference for the development team to enable parallel development and clearly define subsystem boundaries.

Overview of the Software Architecture The proposed system follows a modular architecture to ensure flexibility and efficiency. It is decomposed into three major subsystems:

1. **Data Processing Pipeline:** Responsible for capturing raw network packets and extracting flow-based features.

2. **Machine Learning and XAI Engine:** Utilizes the Isolation Forest algorithm for anomaly detection and SHAP (SHapley Additive exPlanations) for generating human-readable decision explanations.
3. **Backend and User Interface:** A FastAPI-based REST API that connects the database and ML engine to a web-based monitoring dashboard for security analysts.

Design Goals and Non-Functional Requirements As specified in the Requirements Analysis Document (RAD), the primary design goal is to create a transparent, accessible, and resource-efficient security tool. The architecture is specifically designed to satisfy the following non-functional requirements:

- **Performance (Lightweight Design):** The system is optimized for low CPU and memory usage, ensuring low inference latency without requiring expensive enterprise-grade hardware.
- **Usability (Explainability):** By integrating an Explainable AI (XAI) layer, the system moves away from "black-box" AI, allowing analysts to easily understand why a specific traffic flow was flagged.
- **Maintainability:** The strict separation between data extraction, AI prediction, and the user interface allows any module to be updated or replaced without breaking the rest of the system.
- **Legal and Privacy Constraints:** The architecture is designed to extract only flow-based behavioral statistics (e.g., packet counts, byte rates) rather than storing sensitive user payload data.

Traceability and References This SDD is directly traceable to the project's initial Proposal and the Requirements Analysis Document (RAD). The architectural decisions made here are constrained by the need to deploy the prototype on a local server environment. Additionally, the design references the structure of publicly available network security datasets, such as CICIDS2017 and UNSW-NB15, which will be used for model training and evaluation.

1.1 Purpose of the System

The primary purpose of this project is to design and develop an AI-Driven Lightweight Network Intrusion Detection System that can detect abnormal behaviors in network traffic using unsupervised machine learning. Unlike traditional security systems that rely on predefined rules and known attack signatures, our system aims to learn what "normal" network traffic looks like. By doing this, it can adapt to unknown threats and detect new types of cyber attacks by identifying deviations from normal patterns. The system will actively capture real network traffic, extract meaningful behavioral features, and calculate anomaly scores.

A major goal of this system is to solve the "black-box" problem commonly found in artificial intelligence solutions, where users do not understand how the AI makes its decisions. To achieve this, we are integrating an Explainable AI (XAI) layer using SHAP analysis. The purpose of this feature is not only to detect an anomaly but also to clearly explain exactly *why* a specific network flow was considered suspicious.

Furthermore, the system is purposefully designed to have a lightweight and modular architecture. Instead of building a highly complex and resource-heavy enterprise platform, the

purpose is to create an efficient, transparent, and easy-to-use security tool. This approach specifically addresses the needs of small and medium-sized organizations that require reliable cybersecurity but do not have large, dedicated security teams.

1.2 Design Goals

The design goals for the proposed system are directly derived from the non-functional requirements and quality attributes specified in the Requirements Analysis Document (RAD). To satisfy these goals, our high-level architectural design implements the following structural decisions:

Performance and Efficiency (Lightweight Design)

- **Requirement:** The system must operate with computational efficiency, ensuring low CPU and memory usage and minimal inference latency.
- **Architectural Solution:** To address this, the architecture is designed as an efficient prototype suitable for small and medium-sized networks. Instead of using complex and resource-intensive models, the machine learning subsystem utilizes the Isolation Forest algorithm. Additionally, the feature extraction pipeline is designed to transform raw packet data into simple flow-based feature vectors, avoiding heavy processing loads and ensuring the lightweight nature of the system.

Usability and Transparency (Explainability)

- **Requirement:** The system must improve trust, interpretability, and usability, especially for organizations without large security teams.
- **Architectural Solution:** The design addresses this by specifically integrating an Explainable AI (XAI) layer using SHAP analysis into the architecture. The system is structured so that after the anomaly detection model assigns a score, the data passes through the XAI layer to calculate feature contributions. The backend API then delivers this explanation data to the Web Interface, ensuring the dashboard displays real-time, human-readable explanations alongside the alerts.

Maintainability and Modularity

- **Requirement:** The system must be easy to update, test, and maintain over time.
- **Architectural Solution:** To satisfy this, the system consists of multiple interacting components forming a strict modular architecture. The design cleanly separates the packet capture module, the feature engineering pipeline, the anomaly detection model, and the dashboard. A REST API backend built with FastAPI exposes web services to handle communication between these separate layers. This decoupling ensures that developers can modify the machine learning model without affecting the user interface.

Privacy and Legal Compliance

- **Requirement:** The system must monitor local network traffic safely without violating data privacy laws.
- **Architectural Solution:** The system captures and analyzes network traffic data, but the architecture is strictly designed to extract statistical, flow-based features such as packet count, byte rate, flow duration, and port distribution. By focusing on behavioral deviations rather than reading sensitive user payloads through deep packet inspection, the design inherently protects user privacy.

Reliability and Persistence

- **Requirement:** The system must continuously log threats and not lose data if the dashboard is closed.
- **Architectural Solution:** The architecture includes a dedicated database component. Detection results, anomaly scores, and logs are automatically routed by the backend service and stored in the database for persistence and further analysis. This ensures that historical threat data is always available for the analysts.

1.3 Definitions, Acronyms, and Abbreviations

This subsection provides the definitions of all domain-specific terms, acronyms, and abbreviations required to properly interpret this System Design Document.

- **AI (Artificial Intelligence):** The simulation of human intelligence processes by machines, used in this system to detect anomalous network behavior.
- **CICIDS2017 / UNSW-NB15:** Publicly available network traffic datasets used to test and evaluate the intrusion detection model.
- **FastAPI:** A modern web framework for building RESTful APIs in Python, used as the backend service handling communication between the machine learning module, the database, and the user interface.
- **IDS (Intrusion Detection System):** A system that monitors network traffic for suspicious activity or policy violations.
- **Isolation Forest:** An unsupervised machine learning algorithm used as the primary anomaly detection model to learn normal network behavior and identify deviations.
- **REST API:** Representational State Transfer Application Programming Interface, used to expose backend services and handle interactions between the system layers.
- **ROC (Receiver Operating Characteristic):** A performance evaluation metric curve used to analyze the trade-offs between detection accuracy and false positive rates.
- **SHAP (SHapley Additive exPlanations):** A mechanism used to provide explanations for the AI's predictions by highlighting the most influential features contributing to an anomaly decision.
- **SIEM (Security Information and Event Management):** Large, complex, enterprise-grade security platforms that this lightweight and modular prototype aims to provide a simpler alternative to.
- **SYN / ACK:** Network protocol flags representing connection attempts and acknowledgments; their ratio is used as a flow-based metric in the feature extraction pipeline.

- **XAI (Explainable AI):** A layer integrated into the system to enhance interpretability, transparency, and explain why a specific traffic instance is classified as anomalous.

2. Current Software Architecture

Our proposed project does not replace a specific, existing software system within a particular organization. Instead, it addresses the limitations of the current standard solutions used for network security, particularly in small and medium-sized networks.

Currently, the standard approach relies heavily on traditional signature-based intrusion detection systems. These existing systems operate based on predefined rules and known attack patterns. While this traditional method can catch familiar malicious activities, it is becoming increasingly ineffective against zero-day attacks and rapidly evolving cyber threats.

Furthermore, the alternative current solutions are large enterprise-grade SIEM (Security Information and Event Management) or highly complex IDS platforms. These existing solutions are often highly resource-intensive and lack transparency, acting as "black-boxes" that focus only on prediction accuracy without explaining their decisions. For small-scale organizations without large security teams, managing these complex, resource-heavy, and unexplainable current systems represents a significant operational burden. Therefore, the current ad-hoc solution for many small networks is to either use inadequate signature-based tools or leave their networks partially unmonitored.

3. Proposed Software Architecture

The proposed AI-Driven Lightweight Network Intrusion Detection System is designed with a highly modular architecture. This architectural design ensures continuous and efficient interaction between the data processing, machine learning, storage, and visualization layers. The system is logically decomposed into the following interconnected subsystems:

- **Data Capture and Preprocessing Module:** This foundational layer is responsible for capturing raw network traffic data directly from the local network environment. It features a dedicated feature engineering pipeline that actively processes raw packet data and transforms it into structured, flow-based feature vectors.
- **Machine Learning and XAI Subsystem:** The generated feature vectors are passed to this intelligence layer. Here, an anomaly detection model (utilizing the Isolation Forest algorithm) evaluates the network traffic and produces anomaly scores based on deviations from normal behavior, rather than relying on predefined signatures. Crucially, this subsystem also integrates an Explainable AI (XAI) layer using SHAP analysis. This component calculates feature contributions to explain exactly why a specific traffic instance is classified as anomalous.
- **Backend API and Storage Layer:** A REST API backend, developed using FastAPI, serves as the communication bridge for the entire system. It exposes RESTful Web Services that handle the smooth transfer of data between the machine learning module, the database, and the frontend. A dedicated database safely stores all detection results, anomaly scores, and threat logs for persistence and historical analysis.

- **Web Interface (Dashboard):** The presentation layer consists of a monitoring dashboard. This interface continuously communicates with the backend API to retrieve real-time alerts, risk scores, and the SHAP-based visual explanations, presenting them intuitively to the security analyst.

3.1 Overview

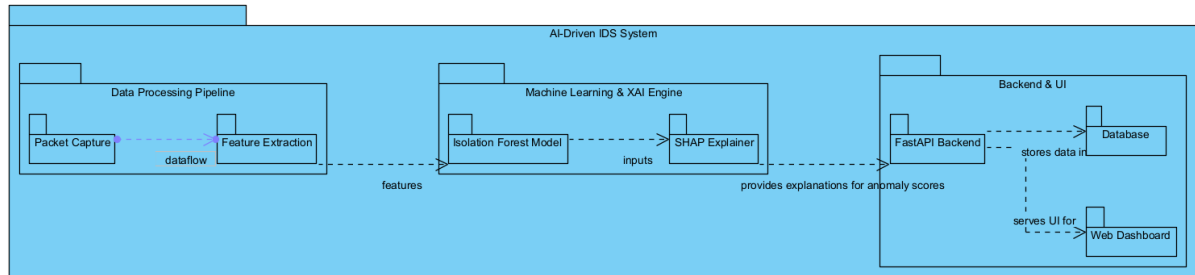


Figure 1.

3.2 System Decomposition

The proposed AI-Driven Lightweight Network Intrusion Detection System is decomposed into multiple interacting subsystems to ensure modularity, maintainability, and parallel development. Each subsystem is designed with a clear responsibility and well-defined interfaces, allowing independent development and scalability.

The system is divided into the following primary subsystems:

1. Data Processing Pipeline

This subsystem is responsible for capturing raw network traffic and transforming it into structured, flow-based feature vectors. It includes components such as packet capture and feature extraction. The main goal of this subsystem is to efficiently preprocess network data while maintaining a lightweight design.

2. Machine Learning and XAI Engine

This subsystem performs anomaly detection and generates interpretable explanations. The anomaly detection is carried out using the Isolation Forest algorithm, which identifies deviations from normal network behavior. In addition, the Explainable AI (XAI) component, implemented using SHAP, provides feature-level explanations for each anomaly score, improving system transparency and usability.

3. Backend and Storage Layer

The backend subsystem, implemented using FastAPI, acts as the central communication hub of the system. It manages the interaction between the machine learning engine, the database,

and the user interface. It is responsible for processing incoming requests, storing detection results, and serving data to the frontend.

4. Web Interface (Dashboard)

This subsystem provides a user-friendly interface for security analysts. It visualizes anomaly scores, alerts, and explanations in real-time. The dashboard retrieves data from the backend via RESTful APIs and presents it in an intuitive and accessible format.

Each of these subsystems is represented as a separate component in the component diagram provided below. The clear separation of concerns ensures that changes in one subsystem (e.g., updating the machine learning model) do not negatively affect other parts of the system.

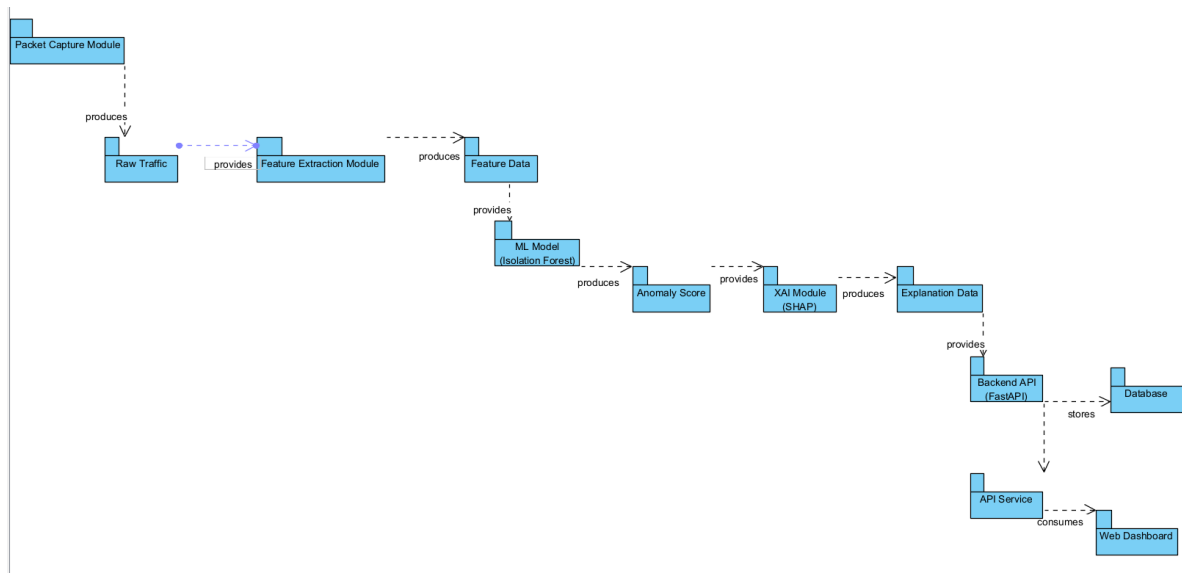


Figure 2.

3.3 Hardware/Software Mapping

The proposed system is deployed using a lightweight and modular architecture where software components are mapped onto a minimal hardware configuration. The system is primarily designed to operate on a single-machine environment for prototype development, but it is also adaptable to distributed deployment scenarios if needed.

The main software components of the system are implemented using Python-based technologies. The backend service is developed using FastAPI, which handles communication between the machine learning module, the database, and the user interface. The anomaly detection model (Isolation Forest) and the XAI component (SHAP) are executed within the same application environment to minimize communication overhead and ensure efficient processing.

From a hardware perspective, the system can operate on a standard personal computer without requiring specialized infrastructure. The application server hosts the backend API and the

machine learning engine, while the database (SQLite or PostgreSQL) is deployed either on the same machine or on a separate node depending on scalability requirements. The user interacts with the system through a web browser, which communicates with the backend via HTTP requests.

In a distributed deployment scenario, the system components can be separated across multiple nodes. For example, the database can be hosted on a dedicated database server, and the backend API can be deployed on a cloud-based server to support multiple users. This separation improves scalability, fault tolerance, and performance under higher workloads.

The mapping of subsystems to hardware and software components is illustrated in the deployment diagram provided below.

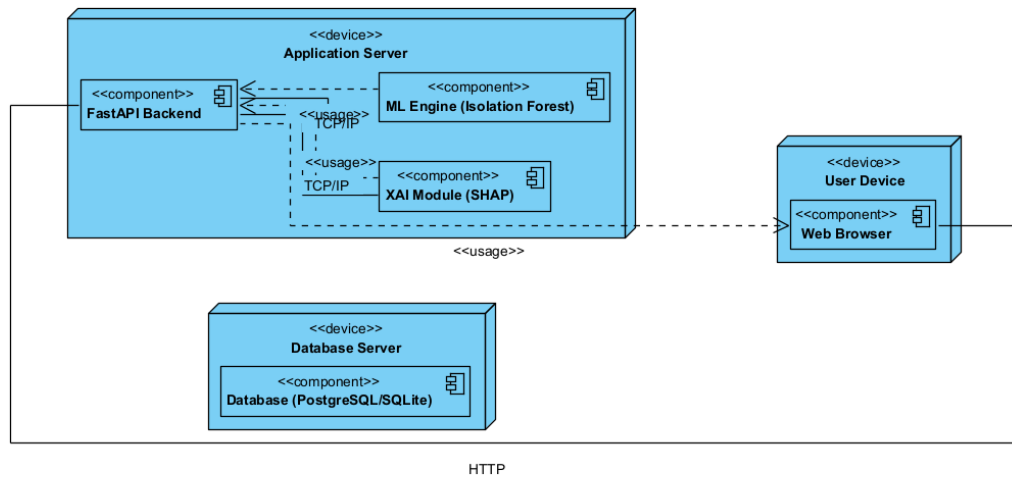


Figure 3. deployment diagram

3.4 Persistent Data Management

The system stores persistent data related to network traffic analysis, anomaly detection, and system logs. The primary goal of persistent data management is to ensure reliable storage, fast retrieval, and historical analysis of detected anomalies.

The main types of persistent data stored in the system are as follows:

- *Flow Features: Structured feature vectors extracted from raw network traffic (e.g., packet count, byte rate, flow duration, port distribution).*
- *Anomaly Scores: Numerical scores generated by the machine learning model indicating the likelihood of abnormal behavior.*

- *Explanation Data: Feature-level explanations generated by the XAI module (SHAP) that describe why a specific flow is classified as anomalous.*
- *Alert Logs: Historical records of detected anomalies, including timestamps and severity levels.*

The system uses a relational database (SQLite for local deployment and PostgreSQL for scalable environments). The database schema is designed to store structured data efficiently and support query operations for analysis and visualization.

The database is encapsulated behind the backend API, meaning that all read/write operations are handled through the FastAPI service. This abstraction ensures data integrity, security, and maintainability by preventing direct access to the database layer.

3.5 Access Control and Security

The system implements a role-based access control model to regulate access to data and system functionality. The primary actors in the system are system administrators and security analysts.

Access rights are defined as follows:

- *Administrator: Full access to all system components, including configuration, data management, and system monitoring.*
- *Security Analyst: Read-only access to anomaly alerts, logs, and explanations through the dashboard interface.*

Authentication is handled using a token-based mechanism (e.g., JWT), ensuring that only authorized users can access the system. All communication between the client and the backend is secured using HTTPS to protect data in transit.

Sensitive data is not stored in raw form. Instead, the system operates on flow-based statistical features, avoiding deep packet inspection and protecting user privacy. Additionally, database access is restricted to the backend service, and credentials are securely managed within the server environment.

3.6 Global Software Control

The system follows a request-driven and event-based control flow. User interactions are initiated through the web interface, which sends HTTP requests to the backend API.

The global control flow is defined as follows:

1. Network traffic is captured and processed by the data processing subsystem.
2. Extracted features are forwarded to the machine learning model for anomaly detection.

3. The model generates anomaly scores, which are then passed to the XAI module.
4. The XAI module produces explanation data for each anomaly.
5. The backend API stores the results in the database and serves them to the web interface.

Concurrency is handled at the backend level, where multiple requests from users can be processed simultaneously using asynchronous API handling. The system ensures that data processing and model inference do not block user interactions.

Subsystem synchronization is achieved through well-defined API calls, ensuring loose coupling and consistent communication between components.

3.7 Boundary Conditions

The system defines clear boundary conditions for initialization, shutdown, and error handling to ensure reliable operation.

- **Startup:** When the system starts, the backend service initializes the database connection and loads the machine learning model into memory. The system then begins monitoring network traffic.
- **Shutdown:** During system shutdown, all active processes are terminated gracefully, and any pending data is saved to the database to prevent data loss.
- **Error Handling:** The system includes mechanisms to handle runtime errors. If the machine learning module fails, the system logs the error and continues operation without interrupting the entire service. In case of database connection failure, retry mechanisms are triggered. All errors are logged for debugging and monitoring purposes.

These boundary conditions ensure system stability and continuous operation under various scenarios.

4. Subsystem Services

Each subsystem provides a set of services that are accessible to other subsystems through well-defined interfaces.

Data Processing Subsystem:

- `capture_packets()`: Captures raw network traffic data.
- `extract_features()`: Transforms packet data into structured feature vectors.

Machine Learning and XAI Subsystem:

- `predict_anomaly()`: Calculates anomaly scores using the Isolation Forest model.
- `generate_explanation()`: Produces feature-level explanations using SHAP.

Backend API Subsystem:

- `store_results()`: Saves anomaly scores and logs into the database.
- `get_alerts()`: Retrieves anomaly alerts for the dashboard.
- `get_explanations()`: Provides explanation data to the user interface.

Database Subsystem:

- `persist_data()`: Stores structured data.
- `query_data()`: Retrieves historical data for analysis.

Web Interface Subsystem:

- `display_alerts()`: Visualizes anomaly alerts.
- `display_explanations()`: Shows explanation data to the user.

These services define the interactions between subsystems and ensure modular communication within the system architecture.

5. References

1. F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in 2008 IEEE International Conference on Data Mining, Pisa, Italy, 2008, pp. 413–422.
2. S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems (NeurIPS), 2017.
3. I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in Proceedings of the International Conference on Information Systems Security and Privacy (ICISSP), 2018.
4. N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in 2015 Military Communications and Information Systems Conference (MilCIS), 2015.
5. R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in 2010 IEEE Symposium on Security and Privacy, pp. 305–316.
6. M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," ACM SIGMOD Record, vol. 29, no. 2, pp. 93–104, 2000.
7. Scikit-learn Developers, "Isolation Forest," [Online]. Available: <https://scikit-learn.org> . [Accessed: Feb. 2026].

8. Zeek Network Security Monitor, “Zeek Documentation,” [Online]. Available: <https://zeek.org> . [Accessed: Feb. 2026].
9. S. Lundberg, “SHAP Documentation,” [Online]. Available: <https://shap.readthedocs.io> . [Accessed: Feb. 2026].